KENNESAW STATE
UNIVERSITY

CS 7357
NEURAL NETS AND DEEP LEARNING

ASSIGNMENT 4

QUANTUM MACHINE LEARNING LAB M2

INSTRUCTOR

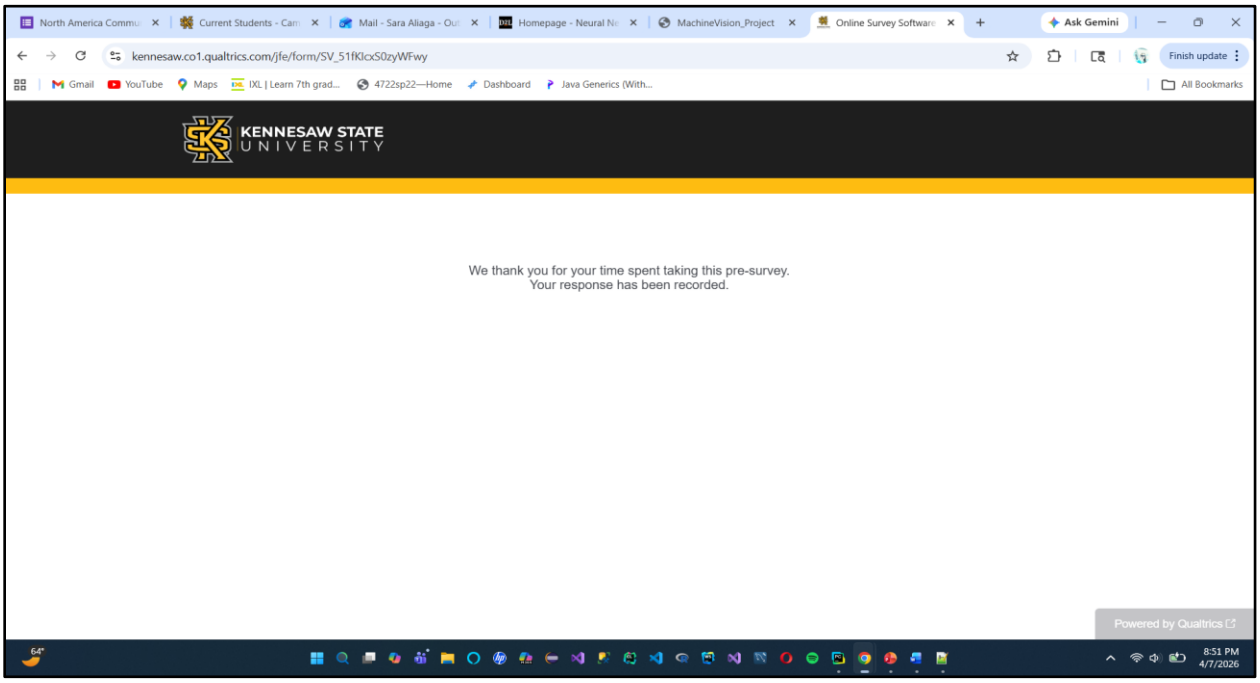
Dr. Zongxing Xie

Sara Aliaga
000835610

Introduction

This lab focuses on the implementation and comparison of Classical Neural Networks (NN) and Quantum Neural Networks (QNN) using a phishing dataset. The objective is to understand how quantum machine learning models perform in a real-world classification problem compared to traditional deep learning approaches. In this lab, we preprocess the dataset, train both models, and evaluate their performance using accuracy and loss metrics. Through this process, we gain insight into the strengths and limitations of quantum models and how they compare them to classical neural networks in practical applications.

Pre – Survey Lab M2 (Screenshot)



1.- Run Install packages

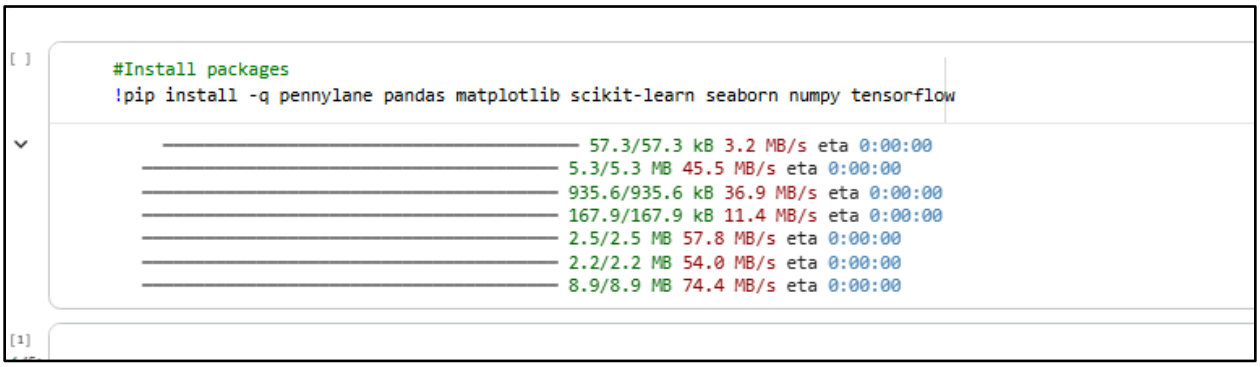


Figure 1. *Installation of required Python packages in Google Collab, including PennyLane, TensorFlow, NumPy, Pandas, and Scikit-learn for machine learning and quantum computing tasks.*

2.– Import packages

```
# Import packages
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

import tensorflow as tf

import pennylane as qml
import pennylane.numpy as qnp

from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.feature_selection import f_classif, SelectKBest
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    recall_score,
    precision_score,
    f1_score,
)

print("TensorFlow version:", tf.__version__)
print("PennyLane version:", qml.__version__)
```

```
TensorFlow version: 2.19.0
PennyLane version: 0.44.1
```

Figure 2. *Importing essential libraries for data manipulation, visualization, classical neural networks, and quantum neural networks.*

3.- Mount to Google Drive

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

import os

# dataset in Drive
data_read_path = os.path.join(
    "/content/drive", "MyDrive", "Colab Notebooks", "Datasets", "Phishing_Legitimate_full.csv"
)

#data_read_path = os.path.join(
 #   "drive", "MyDrive", "Colab Notebooks", "Datasets", "Phishing_Legitimate_full.csv"
#)
print("Dataset path:", data_read_path)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Dataset path: /content/drive/MyDrive/Colab Notebooks/Datasets/Phishing_Legitimate_full.csv
```

Figure 3. *Mounting Google Drive to access the phishing dataset stored in the Collab environment*

4.- Load the Phishing dataset and inspection using pandas

```
# phishing dataset and inspection

phishing_data = pd.read_csv(data_read_path).dropna()

# 5 random samples
display(phishing_data.sample(5))

phishing_data.info()
```

	id	NumDots	SubdomainLevel	PathLevel	UrlLength	NumDash	NumDashInHostname	AtSymbol	TildeSymbol	NumUnderscore
7580	7581	2	1	3	91	5	0	0	0	0
7985	7986	2	1	6	177	18	0	0	0	0
6815	6816	1	0	2	72	1	1	0	0	0
867	868	1	0	3	62	0	0	0	0	0
6991	6992	3	1	1	63	0	0	0	0	0

5 rows × 50 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 50 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   id                          10000 non-null  int64
 1   NumDots                     10000 non-null  int64
 2   SubdomainLevel              10000 non-null  int64
 3   PathLevel                   10000 non-null  int64
 4   UrlLength                   10000 non-null  int64
 5   NumDash                     10000 non-null  int64
 6   NumDashInHostname           10000 non-null  int64
 7   AtSymbol                    10000 non-null  int64
 8   TildeSymbol                 10000 non-null  int64
 9   NumUnderscore               10000 non-null  int64
 10  NumPercent                  10000 non-null  int64
 11  NumQueryComponents          10000 non-null  int64
 12  NumAmpersand                10000 non-null  int64
 13  NumHash                     10000 non-null  int64
```

Figure 4*. Loading the phishing dataset using pandas and displaying sample records along with dataset information for inspection.*

5.- Clean data by handling missing values - Feature selection and scaling for NN and QNN

```
Selected feature indices: [ 0  4 24 26 29 33 34 38 39 47]
```

	NumDots	NumDash	NumSensitiveWords	PctExtHyperlinks	InsecureForms	PctNullSelfRedirectHyperlinks	FrequentDomainNameMismatch	SubmitInfoToEmail	IframeOrFrame	PctExtNullSelfRedirectHyperlinksRT
0	3	0	0	0.000	1	0.0	0	0	0	1
1	3	0	1	0.000	1	0.0	0	0	0	1
2	3	0	0	0.375	1	0.0	0	0	0	0
3	3	1	0	1.000	1	0.0	1	1	0	-1
4	3	0	0	1.000	0	0.0	1	0	1	-1

```
NN train shape: (7000, 10)
NN test shape: (3000, 10)
QNN train shape: (7000, 6)
QNN test shape: (3000, 6)
```

Figure 5. *Data preprocessing including removal of unnecessary columns, feature selection using ANOVA F-test, and scaling for both classical and quantum neural networks*

What does this code do?

Removal of the 'id' column, selection of 'CLASS_LABEL' as the target variable, selection of the top 10 relevant features using ANOVA, normalization for classical neural networks, scaling to [0, 2π]for quantum neural networks, and selection of 6 features corresponding to the 6-qubit quantum circuit

6.- Classical Neural Network (NN) – Implementation

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 256)	2,816
dense_5 (Dense)	(None, 128)	32,896
dense_6 (Dense)	(None, 64)	8,256
dense_7 (Dense)	(None, 1)	65

```
Total params: 44,033 (172.00 KB)
Trainable params: 44,033 (172.00 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/5
186/186 - 2s - 12ms/step - accuracy: 0.7548 - loss: 0.4660 - val_accuracy: 0.8581 - val_loss: 0.2918
Epoch 2/5
186/186 - 1s - 3ms/step - accuracy: 0.8511 - loss: 0.3072 - val_accuracy: 0.8581 - val_loss: 0.3292
Epoch 3/5
186/186 - 1s - 3ms/step - accuracy: 0.8514 - loss: 0.3016 - val_accuracy: 0.8762 - val_loss: 0.2719
Epoch 4/5
186/186 - 1s - 3ms/step - accuracy: 0.8538 - loss: 0.2957 - val_accuracy: 0.8705 - val_loss: 0.2814
Epoch 5/5
186/186 - 1s - 5ms/step - accuracy: 0.8587 - loss: 0.2935 - val_accuracy: 0.8705 - val_loss: 0.2677
```

Figure 6*. Classical Neural Network architecture and training performance showing accuracy and loss improvement over epochs.*

Architecture and training results of the Classical NN model.

The model consists of multiple dense layers (256, 128, 64, and 1 neuron) with a total of 44,033 trainable parameters. The training process shows progressive improvement in accuracy and reduction in loss over 5 epochs, achieving a validation accuracy of approximately 87%.

Classical Neural Network Performance

Accuracy Graph – Classical NN	Loss Graph – Classical NN

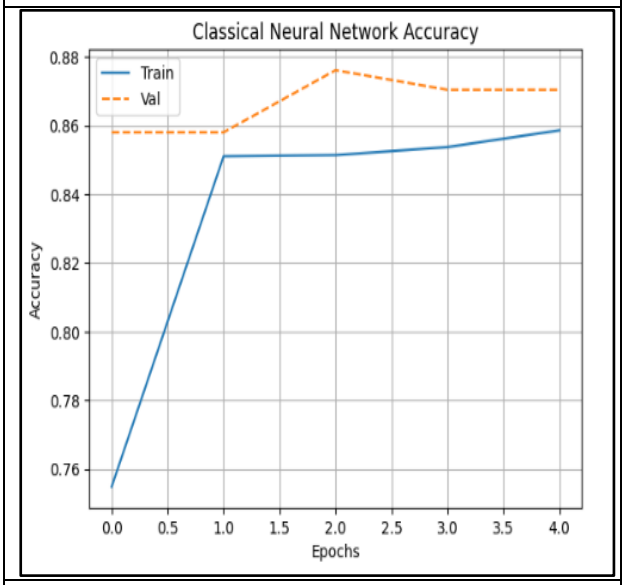
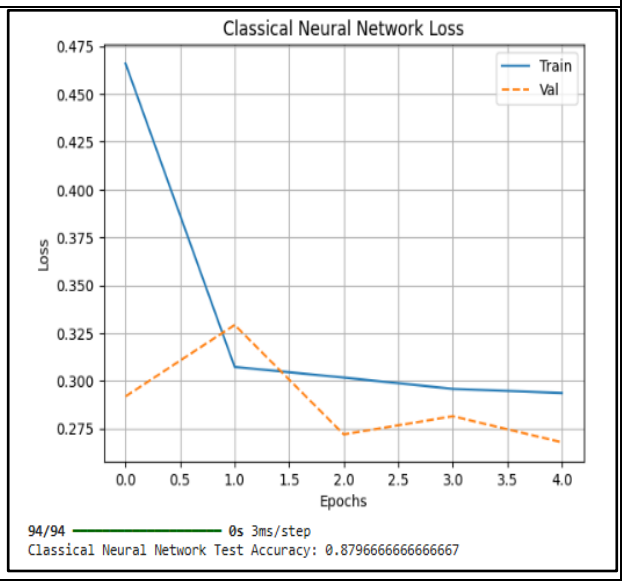

	

Figure 7. *Architecture of the classical neural network consisting of multiple dense layers with sigmoid activation for binary classification*	***Figure 8.*** *Training and validation accuracy of the classical neural network across epochs, showing the model's learning performance.*

Analysis – Classical Neural Network Performance

The accuracy and loss graphs of the Classical Neural Network demonstrate effective learning behavior during training. From the accuracy graph, the training accuracy increases significantly from approximately 75% to around 86% within the first few epochs and then stabilizes. Similarly, the validation accuracy remains consistently high, reaching close to 88%, which indicates that the model generalizes well to unseen data.

The loss graph shows a rapid decrease in training loss during the initial epoch, followed by a gradual stabilization. The validation loss also decreases overall, with minor fluctuations, suggesting that the model is not overfitting and maintains good performance on validation data.

Additionally, the final test accuracy of approximately **87.97%** confirms that the model performs well on the phishing dataset. The close alignment between training and validation curves indicates a well-balanced model with neither underfitting nor significant overfitting. Overall, these results demonstrate that the Classical Neural Network successfully learns the underlying patterns in the dataset and achieves strong classification performance.

7.- Quantum Neural Network Implementation (QNN)

```
Training Quantum Neural Network...
Epoch  1/20 - loss: 0.8737 - val_loss: 0.7404 - train_acc: 0.5358 - val_acc: 0.5267
Epoch  2/20 - loss: 0.6944 - val_loss: 0.6697 - train_acc: 0.5758 - val_acc: 0.5733
Epoch  3/20 - loss: 0.6613 - val_loss: 0.6571 - train_acc: 0.6158 - val_acc: 0.5867
Epoch  4/20 - loss: 0.6520 - val_loss: 0.6511 - train_acc: 0.6217 - val_acc: 0.6033
Epoch  5/20 - loss: 0.6454 - val_loss: 0.6438 - train_acc: 0.6392 - val_acc: 0.6400
Epoch  6/20 - loss: 0.6373 - val_loss: 0.6336 - train_acc: 0.6608 - val_acc: 0.6800
Epoch  7/20 - loss: 0.6280 - val_loss: 0.6230 - train_acc: 0.6950 - val_acc: 0.7000
Epoch  8/20 - loss: 0.6198 - val_loss: 0.6146 - train_acc: 0.7117 - val_acc: 0.7033
Epoch  9/20 - loss: 0.6138 - val_loss: 0.6087 - train_acc: 0.7233 - val_acc: 0.7133
Epoch 10/20 - loss: 0.6096 - val_loss: 0.6047 - train_acc: 0.7350 - val_acc: 0.7333
Epoch 11/20 - loss: 0.6066 - val_loss: 0.6018 - train_acc: 0.7392 - val_acc: 0.7400
Epoch 12/20 - loss: 0.6042 - val_loss: 0.5995 - train_acc: 0.7417 - val_acc: 0.7500
Epoch 13/20 - loss: 0.6020 - val_loss: 0.5973 - train_acc: 0.7442 - val_acc: 0.7600
Epoch 14/20 - loss: 0.6000 - val_loss: 0.5953 - train_acc: 0.7483 - val_acc: 0.7633
Epoch 15/20 - loss: 0.5979 - val_loss: 0.5930 - train_acc: 0.7500 - val_acc: 0.7667
Epoch 16/20 - loss: 0.5956 - val_loss: 0.5905 - train_acc: 0.7575 - val_acc: 0.7733
Epoch 17/20 - loss: 0.5931 - val_loss: 0.5877 - train_acc: 0.7583 - val_acc: 0.7767
Epoch 18/20 - loss: 0.5902 - val_loss: 0.5846 - train_acc: 0.7600 - val_acc: 0.7800
Epoch 19/20 - loss: 0.5870 - val_loss: 0.5810 - train_acc: 0.7667 - val_acc: 0.7933
Epoch 20/20 - loss: 0.5835 - val_loss: 0.5772 - train_acc: 0.7700 - val_acc: 0.7967
```

Picture 9. *Training and validation loss of the quantum neural network, illustrating convergence during optimization*

Quantum Neural Network Accuracy	**Quantum Neural Network Loss**

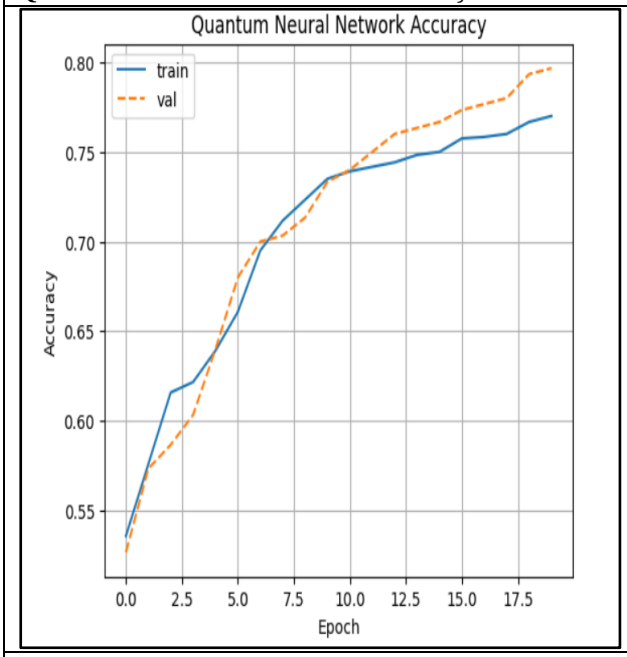
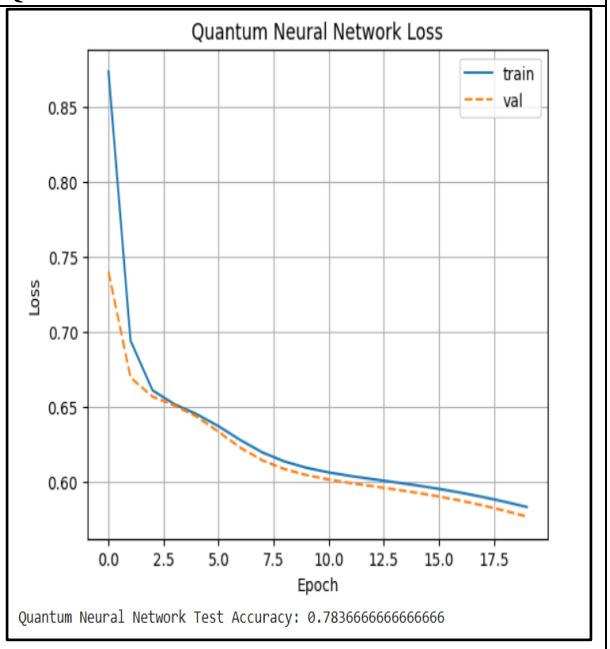

	

Figure 10. *Training and validation accuracy of the quantum neural network across epochs, showing its ability to learn classification patterns.*	***Figure 11.*** *Training loss of the quantum neural network over epochs, demonstrating gradual improvement in model performance.*

Analysis – Quantum Neural Network Accuracy and Loss

The accuracy graph of the Quantum Neural Network shows a steady improvement in both training and validation accuracy over the epochs. The training accuracy increases from approximately 53% to around 77%, while the validation accuracy reaches close to 80%. The close alignment between the training and validation curves indicates that the model generalizes well and does not suffer from significant overfitting.
The loss graph further supports this observation, as both training and validation loss decrease consistently throughout the training process. The rapid drop in loss during the initial epochs indicates that the model quickly learns important features from the dataset. As training progresses, the loss decreases more gradually, showing that the model is converging toward an optimal solution.
Overall, the QNN demonstrates stable learning behavior, with gradual improvements in accuracy and consistent reduction in loss. These results indicate that the model is effectively capturing classification patterns, although at a moderate learning pace.

8.- Comparative Analysis: Classical Neural Network and Quantum Neural Network

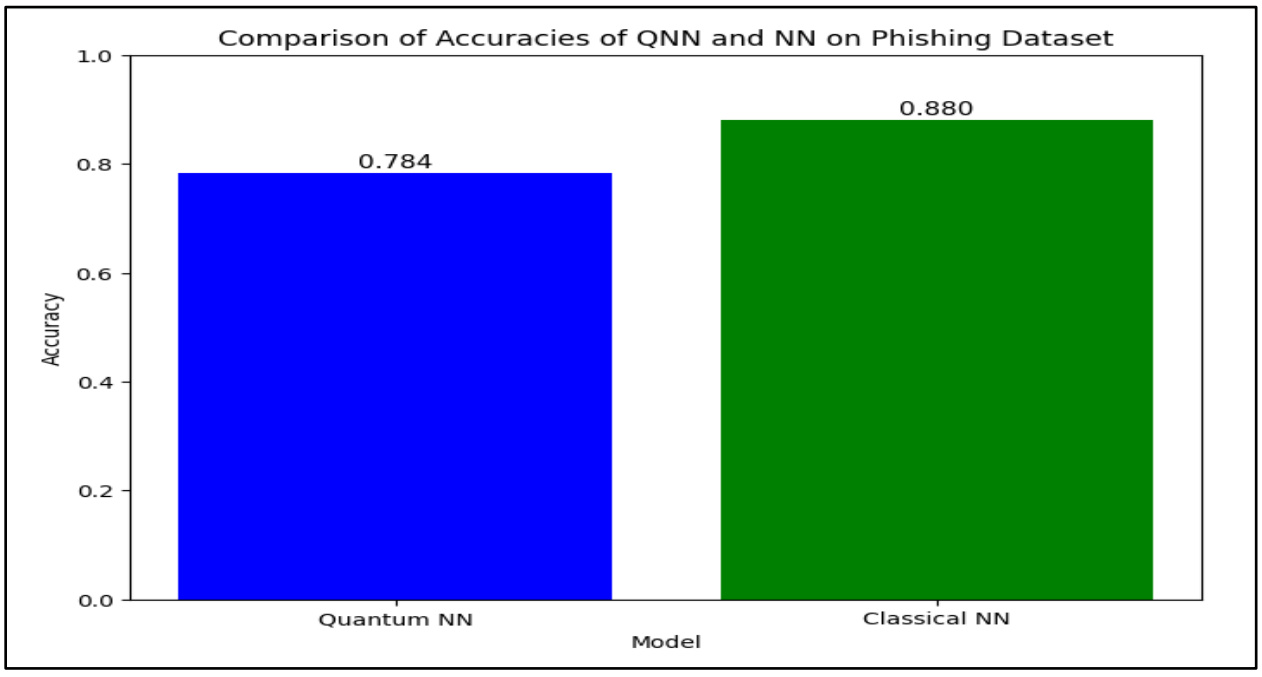


Figure 12. *Comparison of classification accuracy between the classical neural network and the quantum neural network on the phishing dataset.*

Analysis:

The comparison demonstrates that both models perform effectively on the phishing dataset, with the classical neural network achieving slightly higher accuracy due to its greater model capacity and maturity. However, the quantum neural network shows promising results, highlighting the potential of quantum machine learning for future applications.

Results

The performance of both the Classical Neural Network (NN) and the Quantum Neural Network (QNN) was evaluated using the phishing dataset.

The Classical Neural Network achieved a test accuracy of **0.880 (88.0%)**, while the Quantum Neural Network achieved a test accuracy of **0.784 (78.4%)**, as shown in Figure 12. This indicates that the classical model outperformed the quantum model in this classification task.

The training process of the Quantum Neural Network demonstrates steady learning behavior. As observed in the accuracy graph, both training and validation accuracy increased consistently over the epochs, reaching approximately **77–80%** by the end of training. Similarly, the loss graph shows a gradual decrease in both training and validation loss, indicating that the model successfully minimized error and converged during optimization.

However, the Quantum Neural Network exhibited lower overall accuracy compared to the Classical Neural Network. This can be attributed to several factors, including:

- Limited number of qubits (6 features used)
- Smaller training subset (1500 samples)
- Simpler model capacity compared to deep classical architectures

Despite these limitations, QNN still demonstrates meaningful learning capability and produces competitive results.

Conclusion

In this lab, both Classical Neural Networks and Quantum Neural Networks were implemented and evaluated for phishing website classification.

The results show that the Classical Neural Network achieved higher accuracy (**88.0%**) compared to the Quantum Neural Network (**78.4%**). This is expected, as classical models are more mature and can handle larger feature spaces and datasets more efficiently.

Nevertheless, the Quantum Neural Network demonstrated promising performance, successfully learning patterns from the data and achieving a reasonable classification accuracy. The decreasing loss and increasing accuracy trends confirm that the quantum model is capable of effective training and generalization.

This experiment highlights that while quantum machine learning is still in its early stages, it has strong potential for future applications. With advancements in quantum hardware, larger qubit systems, and improved optimization techniques, Quantum Neural Networks could become competitive with classical models.

Overall, this lab provides valuable insight into the comparison between classical and quantum approaches, emphasizing both the strengths of classical methods and the emerging potential of quantum computing in machine learning.

Post – Lab

Post Lab Answers questions: (Lab M2)

1. Comparison of Neural Network and Quantum Neural Network performance

The Classical Neural Network outperformed the Quantum Neural Network in this experiment. The classical model achieved an accuracy of 88.0%, while the quantum model achieved 78.4%. This difference shows that classical models are currently more effective for this type of

classification problem. However, the quantum model still demonstrated reasonable performance and the ability to learn meaningful patterns from the data.

2. Impact of Training Dataset Size on Performance

The size of the training dataset significantly impacts model performance. The Classical Neural Network was trained on a larger portion of the dataset, allowing it to learn more complex patterns and generalize better. In contrast, the Quantum Neural Network was limited to 1500 samples, which restricted its learning capability. Larger datasets typically improve model accuracy, while smaller datasets can lead to underfitting.

3. Limitations and Potential Improvements

The Quantum Neural Network has several limitations:

Limited number of qubits (only 6 features used)
Smaller training dataset
Higher computational cost and slower training
Less mature optimization techniques compared to classical models

Potential improvements include:

Increasing the number of qubits to include more features
Using more advanced quantum circuits
Training on larger datasets
Applying hybrid quantum-classical approaches
Optimizing hyperparameters such as learning rate and number of layers

4. Analysis of QNN vs NN in Practical Applications of Quantum Neural Networks on real-world datasets

In practical applications, Classical Neural Networks are currently more reliable and widely used due to their high performance, scalability, and efficiency. However, Quantum Neural Networks represent an emerging field with significant future potential. Although QNNs are not yet as powerful as classical models, they introduce new computational approaches based on quantum mechanics, such as superposition and entanglement. As quantum hardware improves, QNNs may become more competitive and useful for solving complex real-world problems.

Questions from post-Lab:

(i)What type of data sets do you think are suitable for Quantum Neural Network?

Quantum Neural Networks are most suitable for datasets that are relatively **small to medium in size**, have a **limited number of important features**, and involve **classification or pattern recognition tasks**. This is because current QNNs are constrained by the number of qubits available, so high-dimensional datasets usually need feature reduction before they can be processed. Structured numerical datasets are especially suitable, since they can be scaled and encoded into quantum circuits more easily than large images or text. Therefore, tabular datasets with a smaller number of strong predictors are good candidates for QNN models.

(ii)What adjustments can be made to further improve the performance of Quantum Neural Network?

Several adjustments can improve the performance of a Quantum Neural Network. First, **better feature selection** can help ensure that only the most informative variables are encoded into the quantum circuit. Second, increasing the **number of qubits** would allow the model to process more features, which may improve classification accuracy. Third, the QNN can be improved by tuning hyperparameters such as the **learning rate, batch size, number of layers, and number of training epochs**. Fourth, using hybrid **quantum-classical architecture** may improve performance by combining the strengths of both methods. Finally, improvements in **quantum hardware and simulators** could reduce noise and computational limitations, making QNNs more effective on real-world datasets.

(iii) Try using a different dataset for Quantum Neural Network and compare and analyze the performance between Neural Network and Quantum Neural Network models on this dataset.

For a follow-up experiment, a suitable alternative dataset would be a **small binary classification dataset from Kaggle**, such as diabetes, heart disease, or breast cancer dataset. These datasets are more manageable for QNNs because they usually contain structured numerical features and fewer dimensions than more complex datasets.
When comparing Neural Network and Quantum Neural Network models on a different dataset, the Classical Neural Network would likely still achieve higher accuracy because it can process more features efficiently and is supported by well-developed optimization methods. The Quantum Neural Network may achieve reasonable performance, but it would still be limited by the number of qubits, reduced feature input, and slower training time. However, if the dataset is smaller, cleaner, and has fewer important features, the performance gap between NN and QNN may become smaller.

Overall, the comparison would likely show that **Classical Neural Networks currently perform better on most real-world datasets**, while **Quantum Neural Networks show promising potential**, especially for future applications as quantum computing technology advances. QNNs may become more competitive when applied to carefully selected datasets with lower dimensionality and strong feature relevance.

Post – Lab Survey (Screenshot)